

# **Análisis de Caso de Negocio, Estado del Arte y Benchmark de Productos**

## **Resumen Ejecutivo y Visión de IntellOps**

### **Contexto del proyecto**

El presente documento se enmarca en las actividades científicas y académicas del Grupo de Investigación y Desarrollo Aplicado a Sistemas Informáticos y Computacionales (GIDAS), perteneciente a la Universidad Tecnológica Nacional, Facultad Regional La Plata (UTN FRLP). Específicamente, este informe consolida los fundamentos teóricos para el subproyecto denominado IntellOps (SP-2).

### **Visión General**

La visión estratégica de IntellOps consiste en la conceptualización y construcción de un Producto Mínimo Viable (MVP) —un prototipo funcional de software— orientado a democratizar el monitoreo continuo y el análisis predictivo de infraestructuras tecnológicas. Eludiendo la alta carga financiera de las plataformas comerciales privativas y la enorme complejidad de integración manual del ecosistema puro de código abierto, la solución propone aplicar modelos de Machine Learning de forma nativa sobre Series Temporales para lograr una verdadera detección predictiva de anomalías. Apoyado sobre un agente ligero programado en Python, el sistema permitirá ejecutar inferencia matemática en recursos locales o de borde (Edge Computing), ofreciendo protección a los valiosos clústeres de investigación universitarios sin interferir con sus cargas de trabajo principales.

### **Objetivo del Documento**

El objetivo primordial de este informe es justificar de manera teórica, técnica y analítica la viabilidad y necesidad del desarrollo del MVP de IntellOps, estableciendo los cimientos estratégicos antes de proceder a las fases de diseño arquitectónico e implementación.

## **2. Relevamiento de Herramientas Existentes y posibles soluciones**

Para comprender cómo construir una solución viable y robusta en un entorno de laboratorio académico con recursos escasos, es imperativo diseccionar a nivel técnico el funcionamiento interno ("bajo el capó") de las herramientas hegemónicas del mercado. El objetivo es identificar las metodologías que las hacen exitosas y los diseños que las hacen inalcanzables, para finalmente extraer un patrón arquitectónico que sea aplicable a la propuesta de IntellOps.

### **2.1. Soluciones Comerciales (Datadog, New Relic, Dynatrace): El Paradigma de la "Caja Negra" SaaS**

Las grandes plataformas comerciales operan bajo un modelo centralizado de Software as a Service (SaaS). Su arquitectura se divide fundamentalmente en dos partes: agentes locales propietarios de recolección y un motor analítico masivo en la nube pública.

**Funcionamiento y Características Principales:** Estas herramientas dependen de la instalación de agentes propietarios y altamente acoplados en los servidores del usuario. Estos agentes realizan auto-instrumentación en tiempo de ejecución (por ejemplo, inyectando código en la máquina virtual de Java o interceptando llamadas a nivel del núcleo del sistema operativo usando eBPF) para recolectar trazas, registros y métricas sin intervención manual. Una vez capturados, estos datos son transmitidos obligatoriamente (exfiltrados) hacia los clústeres en la nube del proveedor.

Es en la nube del proveedor donde ocurre el verdadero diferencial tecnológico: la aplicación de AIOps. Utilizan sofisticados clústeres de cómputo para aplicar algoritmos de detección de anomalías contextuales y aprendizaje automático (ML) sobre los datos ingeridos. Por ejemplo, Datadog y Dynatrace despliegan motores causales que analizan millones de métricas por segundo para generar topologías de red automatizadas y líneas base (baselining) dinámicas.

**Alcance y Barreras para Entornos Académicos:** El alcance funcional es sobresaliente, pero el mecanismo operativo es económicamente inviable para un laboratorio universitario público. El problema técnico-financiero radica en que estas empresas cobran por el "derecho informático" de enviar, procesar y retener los datos en sus nubes. Los esquemas de precios de New Relic y Datadog, por ejemplo, imponen cargos que oscilan entre \$0.40 y \$0.60 USD por gigabyte ingerido, además de aplicar trampas de facturación basadas en el "high-water mark" o pico máximo de uso de hosts/contenedores. Un clúster de investigación de tamaño mediano que genere grandes volúmenes de telemetría de red y registros experimentales incurriría en decenas de miles de dólares anuales en costos de retención. Además, los algoritmos de detección son "cajas negras": no pueden ser auditados, modificados ni ejecutados localmente, lo que atenta contra la soberanía de los datos académicos.

## **2.2. Alternativas Open Source (Prometheus + Grafana): El Paradigma Desacoplado y Manual**

El ecosistema Open Source dominado por la Cloud Native Computing Foundation (CNCF) ofrece un enfoque diametralmente opuesto. La dupla de Prometheus (para almacenamiento y consulta) y Grafana (para visualización) reina en las infraestructuras de Kubernetes.

**Funcionamiento y Características Principales:** Prometheus opera mediante una arquitectura de recolección activa o modelo "pull". Utiliza un servidor central que navega regularmente hacia endpoints HTTP expuestos por las aplicaciones y extrae métricas de Series Temporales basándose en un formato de texto simple con etiquetas clave-valor. Su base de datos integrada (TSDB) es extraordinariamente rápida, pero está diseñada casi exclusivamente para retención de corto plazo (días o semanas). El lenguaje de consulta nativo, PromQL, es potente para agregaciones matemáticas instantáneas, pero carece de capacidades complejas de predicción.

Para lograr la detección de anomalías, las integraciones en este entorno dependen típicamente de líneas base estadísticas manuales, como las desviaciones estándar sobre un promedio móvil (e.g., bandas de Bollinger improvisadas mediante PromQL). Sin embargo, la varianza

matemática pura fracasa frente a las fluctuaciones normales de tráfico y genera una avalancha inmanejable de falsos positivos en métricas no estacionarias.

Alcance y Barreras para Entornos Académicos: Si bien el costo de licencias es nulo, el costo computacional y de tiempo de desarrollo (horas-hombre) es prohibitivo. Para llevar este ecosistema al nivel del AIOps comercial, un laboratorio debe:

Exportar flujos masivos de datos fuera de Prometheus utilizando el protocolo `remote_write` hacia bases de datos externas de largo plazo (Thanos, Cortex o InfluxDB).

Levantar infraestructuras separadas para correr modelos de Machine Learning en Python.

Orquestar bucles asíncronos que reinyecten las predicciones al sistema original para poder emitir alertas.

Un laboratorio universitario carece del equipo dedicado de MLOps necesario para sostener y afinar permanentemente esta arquitectura frágil y desarticulada, que además penaliza severamente el consumo de CPU y memoria del clúster destinado a la investigación científica.

### **2.3. Hacia una Solución Factible: La Estrategia Arquitectónica de IntellOps**

Tras analizar detalladamente ambos mundos, resulta claro cómo derivar una solución tecnológica factible y poderosa para instituciones con presupuestos deprimidos: se debe extraer la inteligencia de la nube y compactarla en el borde (Edge Computing) utilizando las herramientas del ecosistema abierto.

Para materializar esta solución sin dilapidar recursos, el diseño de IntellOps amalgama principios de ambos mundos:

Agente Ligero en Python (Telemetría eficiente y local): En lugar de instalar complejos exportadores de Prometheus que exigen una red de recolección continua, o agentes comerciales que exfiltran la data a la nube, se despliega un agente nativo en Python. Este agente, inspirado en el diseño asíncrono de colectores modernos, extrae métricas de las aplicaciones de investigación (uso de memoria, tiempos de CPU, logs de eventos) procesándolas de manera estrictamente local o on-premise. Esto elude de inmediato cualquier costo por ingesta en dólares y preserva la privacidad del experimento.

Inferencia Algorítmica Inteligente pero Frugal: Para obtener el nivel de detección de Datadog sin sus servidores masivos, se implementa un modelo de detección directamente acoplado al colector. Se recurre a Isolation Forest, un algoritmo de partición espacial con una complejidad lineal, para efectuar un filtrado de primera línea identificando picos bruscos y ataques estocásticos con una huella en memoria casi indetectable.

Análisis Secuencial Profundo Controlado: Únicamente sobre los subconjuntos de datos marcados como sospechosos, se activa la inferencia de redes LSTM Autoencoders pre-entrenadas. Las arquitecturas basadas en LSTM comprenden la cronología temporal y las anomalías de tendencia lenta, otorgando la profundidad analítica necesaria pero consumiendo

ciclos de cómputo solo cuando es estrictamente necesario, evitando así la saturación del hardware del laboratorio que debe priorizar sus simulaciones.

Al aplicar de forma local (Edge AI) este conjunto dual de algoritmos sobre una recolección en Python, se logra emular las capacidades predictivas proactivas de las plataformas comerciales de pago, eliminando simultáneamente los costos punitivos de licenciamiento y erradicando el esfuerzo insostenible de integrar ecosistemas fragmentados de código abierto. Esta es la síntesis arquitectónica que hace viable a IntellOps en el marco de la rigidez presupuestaria académica nacional.

### **3. Área de Aspectos de Seguridad y Gobernabilidad del Software**

#### **3.1 Seguridad en el Agente de Captura (Data Collection Security)**

Las plataformas comerciales de observabilidad modernas, como Datadog, Dynatrace y New Relic, basan gran parte de su funcionamiento en agentes instalados directamente sobre los sistemas monitoreados. Estos agentes ejecutan tareas de captura continua de métricas, eventos, registros y trazas, convirtiéndose en el punto de entrada principal de toda la información utilizada posteriormente por los motores de análisis y detección de anomalías.

La literatura especializada sobre sistemas de recolección de datos identifica que la etapa de captura constituye uno de los principales puntos de exposición desde la perspectiva de seguridad. Las Fuentes señalan que los sistemas de adquisición de datos deben proteger simultáneamente la confidencialidad, integridad y disponibilidad de la información recolectada, ya que una alteración o acceso indebido durante esta etapa compromete todo el proceso posterior de análisis.

Asimismo, los estudios sobre monitoreo y analítica de seguridad muestran que los agentes modernos recolectan información proveniente de múltiples capas de la infraestructura, incluyendo métricas de CPU, memoria, procesos, eventos del sistema operativo, tráfico de red y registros de aplicaciones. Esta amplitud de observación incrementa la capacidad de detección, pero también amplía la superficie potencial de exposición si los mecanismos de captura no se encuentran correctamente aislados o restringidos.

Como consecuencia de estos riesgos, las arquitecturas modernas implementan mecanismos de aislamiento y control de privilegios para limitar el alcance operativo de los agentes. En lugar de ejecutar procesos con acceso irrestricto al sistema, la tendencia actual consiste en aplicar el principio de mínimo privilegio, restringiendo el acceso únicamente a los recursos estrictamente necesarios para la observación.

Los ecosistemas cloud-native han reforzado esta estrategia mediante el uso de contenedores, políticas RBAC, ejecución sin privilegios elevados y separación explícita entre los componentes encargados de recopilar telemetría y aquellos responsables de procesarla.

La propuesta arquitectónica de IntellOps se alinea naturalmente con esta tendencia. Al utilizar un agente ligero desarrollado en Python y orientado a la ejecución local, resulta posible limitar la recolección exclusivamente a las métricas necesarias para los algoritmos de detección de anomalías, evitando accesos innecesarios a información sensible del sistema anfitrión.

### 3.2 Seguridad en Tránsito e Ingesta

La transmisión de métricas representa uno de los puntos críticos dentro de cualquier plataforma de observabilidad. Una vez recolectada la información, la arquitectura debe garantizar que los datos lleguen íntegros al sistema de almacenamiento y análisis, evitando modificaciones, interceptaciones o inyecciones de información maliciosa durante el tránsito.

Los textos analizados identifican como amenazas frecuentes el acceso no autorizado a los servidores de recolección, la alteración de los datos transmitidos, los ataques de denegación de servicio y la incorporación de información falsa dentro de la pipeline de monitoreo. Para mitigar estos riesgos, las arquitecturas modernas incorporan mecanismos de cifrado, autenticación y control de acceso sobre los componentes de ingesta. Entre las defensas más utilizadas se encuentran las conexiones cifradas mediante TLS/SSL, los sistemas de autenticación de agentes y las políticas de acceso restringido a los servidores que reciben telemetría.

Las plataformas comerciales de observabilidad implementan canales de comunicación cifrados entre los agentes y los servicios centrales de procesamiento. Este enfoque permite verificar la procedencia de las métricas y garantizar que la información utilizada por los motores analíticos no haya sido modificada durante la transmisión.

La necesidad de estos controles aumenta a medida que crece el volumen de telemetría procesada. Un sistema de detección de anomalías depende directamente de la calidad de los datos de entrada; métricas alteradas o falsificadas degradan la precisión de los modelos y comprometen la confiabilidad de las alertas generadas.

Bajo este escenario, la adopción de mecanismos estándar como TLS para la protección del tráfico y autenticación basada en tokens permite asegurar la integridad y autenticidad de la telemetría utilizando tecnologías maduras, ampliamente documentadas y compatibles con una implementación ligera. Este enfoque proporciona un nivel de seguridad adecuado para los objetivos del MVP sin incrementar significativamente los costos de desarrollo ni la carga operativa de la plataforma.

### 3.3 Gobernabilidad, Calidad y Estándares de Código

La tendencia predominante en la industria consiste en desplazar los controles de calidad hacia etapas tempranas del desarrollo mediante pipelines de Integración Continua (CI). Estos procesos ejecutan pruebas unitarias, análisis estático, validaciones de estilo y controles de calidad sobre cada modificación propuesta, reduciendo la probabilidad de incorporar errores a ramas estables del proyecto.

IntellOps se desarrolla como un proyecto de investigación aplicada con posibilidades de continuidad académica y transferencia tecnológica. Bajo este escenario, la adopción temprana de mecanismos de gobernabilidad permite construir una base de código mantenible sin incrementar significativamente la complejidad del desarrollo.

La utilización de Git como sistema de control de versiones, junto con revisiones mediante Pull Requests, protección de la rama principal y herramientas automáticas de análisis para Python, proporciona un esquema de trabajo alineado con las prácticas observadas en los principales proyectos de observabilidad. Este enfoque mejora la trazabilidad de los cambios, facilita la incorporación de nuevos colaboradores y reduce el riesgo de degradación técnica a medida que evolucione el MVP.

### 3.4 Gestión de Dependencias y Licenciamiento Open Source

Ante la creciente adopción de componentes open source, industria ha incorporado prácticas de auditoría continua orientadas a detectar vulnerabilidades y riesgos asociados a la cadena de suministro de software. Organizaciones como OpenSSF y OWASP promueven mecanismos de monitoreo permanente de dependencias, actualización controlada de paquetes y análisis automatizados de seguridad durante el ciclo de desarrollo.

Respecto al licenciamiento, existe una tendencia marcada en la industria hacia modelos permisivos como MIT y Apache 2.0, particularmente en proyectos vinculados a observabilidad, cloud computing y herramientas para desarrolladores.

La arquitectura propuesta para IntellOps se apoya principalmente en tecnologías open source del ecosistema Python. En consecuencia, la gestión controlada de dependencias constituye un requisito necesario para preservar la estabilidad y reproducibilidad del proyecto a lo largo del tiempo.

La incorporación de herramientas automáticas de auditoría de paquetes permite detectar vulnerabilidades sin demandar recursos adicionales significativos, manteniendo coherencia con las restricciones operativas propias de un proyecto de investigación. En materia de licenciamiento, Apache 2.0 surge como una alternativa adecuada para un eventual proceso de transferencia tecnológica, ya que conserva el carácter abierto del proyecto mientras facilita su adopción futura por otras instituciones académicas u organizaciones interesadas en extender la plataforma.

## 4. Área de Quality Assurance (QA) y Estrategias de Validación

#### 4.1 Estrategia de Validación de Ingesta y Concurrencia

Las plataformas de observabilidad procesan flujos continuos de métricas generadas simultáneamente por múltiples agentes distribuidos. La validación de la pipeline de ingesta busca garantizar consistencia, disponibilidad y tolerancia a fallos bajo condiciones de alta concurrencia, pérdida de paquetes y degradación parcial de la infraestructura.

La industria utiliza pruebas de carga, estrés y resiliencia para identificar cuellos de botella y verificar que la plataforma mantenga niveles aceptables de latencia y disponibilidad. En sistemas basados en series temporales, la detección de pérdidas de datos durante la ingesta constituye uno de los principales criterios de validación.

Aunque IntellOps no persigue escalas equivalentes a las plataformas comerciales, resulta necesario validar que la arquitectura pueda procesar simultáneamente métricas provenientes de distintos agentes sin pérdidas significativas de información. La ejecución de pruebas controladas permitirá evaluar el comportamiento de la pipeline completa dentro de las restricciones de un proyecto de investigación aplicada.

#### 4.2 Benchmarking y Métricas de Calidad en Modelos de ML

La evaluación de algoritmos de detección de anomalías en AIOps requiere métricas capaces de medir adecuadamente la identificación de eventos poco frecuentes. Precision, Recall y F1-Score se han consolidado como los indicadores más utilizados para evaluar la calidad de modelos aplicados a series temporales.

Las plataformas modernas priorizan la reducción de falsos positivos debido al impacto operativo que generan sobre los equipos responsables del monitoreo. Como consecuencia, los modelos suelen complementarse con mecanismos de correlación de eventos, análisis contextual y umbrales dinámicos para mejorar la precisión de las alertas.

La validación de los modelos propuestos deberá apoyarse principalmente en Precision, Recall y F1-Score, permitiendo comparar objetivamente diferentes enfoques de detección. La reducción de falsas alertas constituye un criterio prioritario debido a que el objetivo del sistema es asistir la toma de decisiones y no sustituir completamente la supervisión humana.

### 4.3 Estrategias de Simulación de Fallas e Inyección de Errores

Chaos Engineering propone introducir fallos controlados dentro de una arquitectura para validar su capacidad de detección, respuesta y recuperación. El enfoque busca reproducir escenarios reales de degradación mediante eventos como sobrecarga de CPU, consumo excesivo de memoria, interrupciones de servicios o problemas de conectividad.

Las organizaciones que operan infraestructuras críticas utilizan pruebas destructivas controladas para verificar que sus sistemas continúen detectando eventos relevantes y generando alertas aún bajo condiciones adversas. Estas prácticas permiten identificar debilidades que normalmente no aparecen durante las pruebas funcionales tradicionales.

El entorno de laboratorio permite reproducir versiones simplificadas de estos escenarios mediante generación artificial de carga, consumo de recursos o interrupción de servicios monitoreados. Estas pruebas proporcionarán conjuntos de datos controlados para validar el comportamiento de la plataforma desde la captura de métricas hasta la generación de alertas.



## Referencias

- **Aguerre Guerisoli, F. (2025).** *Desarrollo de una prueba de concepto de prácticas AIOps*. Repositorio Digital RAD.  
<https://rad.ort.edu.uy/handle/20.500.11968/7773>. (Este es un excelente proyecto académico en español que detalla la instrumentación de una arquitectura con OpenTelemetry, la integración de Prometheus/Grafana y el uso del algoritmo Isolation Forest).
- **Rathor, G. (2025).** Observability: anomaly detection at scale with prometheus. *International Journal of Computational and Experimental Science and Engineering*, 11(4).  
<https://www.ijcesen.com/index.php/ijcesen/article/view/4576>.
- **Sanches, J., & Pereira, P. R. (2026).** Network and Systems Monitoring with Prometheus and Grafana. En A. Rocha, C. J. Costa, F. García Peñalvo, & R. Gonçalves (Eds.), *Proceedings of 20th Iberian Conference on Information Systems and Technologies (CISTI 2025) - Volume 1* (pp. 367-378). Springer.  
[https://doi.org/10.1007/978-3-032-10929-3\\_32](https://doi.org/10.1007/978-3-032-10929-3_32).
- **Shaheen, Q. J., & Alomari, E. S. (s.f.).** A novel anomaly detection-based hybrid model for the prediction of ransomware attacks by monitoring network traffic. *International Journal of Advanced Science and Information Systems*.  
<https://xlscience.org/index.php/IJASIS/article/view/1442>.
- **Zhong, Z., Fan, Q., Zhang, J., Ma, M., Zhang, S., Sun, Y., Lin, Q., Zhang, Y., & Pei, D. (2023).** A Survey of Time Series Anomaly Detection Methods in the AIOps Domain. *arXiv preprint arXiv:2308.00393*.  
<https://doi.org/10.48550/arXiv.2308.00393>.
- Cobb, C., Sudar, S., Reiter, N., Anderson, R., Roesner, F., & Kohno, T. (2018). *Computer Security for Data Collection Technologies*. *Development Engineering*, 3, 1–11.  
<https://www.sciencedirect.com/science/article/pii/S2352728516300677>
- Jing, X., Yan, Z., Pedrycz, W., & Chen, L. (2019). *Security Data Collection and Data Analytics in the Internet: A Survey*. *IEEE Communications Surveys & Tutorials*, 21(1), 586–618.  
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8428412>
- GitHub. (s.f.). *GitHub Documentation*. <https://docs.github.com>
- OpenTelemetry Authors. (s.f.). *OpenTelemetry Documentation*.  
<https://opentelemetry.io/docs/>
- OWASP Foundation. (s.f.). *OWASP Foundation*. <https://owasp.org>
- Rathor, G. (2025). Observability: Anomaly Detection at Scale with Prometheus.  
<https://www.ijcesen.com/index.php/ijcesen/article/view/4576>

- Basiri, A., et al. (2017). Chaos Engineering. <https://arxiv.org/abs/1702.05843>